專題研究(一)進度報告-進度2

專題名稱：視覺化網路攻擊自動化分析平台
成員：D1378300 胡安硯、D1300043 何昇駿、D1256832 石任立

一、工作與進度報告

(本次進度 :模型訓練與重建誤差閾值調校)

本次進度主要更新內容 :

- 加入 Django 框架：完成 REST API 端點設計與實作(api/urls.py、api/views.py)，提供 Session 管理、即時監控、結果分析等前後端溝通介面(目前僅是雛形，之後會做更全面的實作)。
- 模型訓練相關程式實作：完成 CNN Autoencoder 訓練流程(run_training.py)以及重建誤差閾值調校核心邏輯。
- 加入半監督式學習：新增 semi_supervised_trainer.py，訓練策略採用邊界損失(Margin Loss) + 重建損失

目前的程式架構圖

```
final_project/
    │
    ├── core/
    │   ├── parser.py / config.py / anomaly_detector.py
(封包解析、配置、規則偵測)
    │   ├── capture.py / pcap_analyzer.py
(即時抓包、離線 PCAP 分析)
    │   ├── packet_visualizer.py / dataset_builder.py
(封包影像化、資料集建構)
    │   ├── dataset_loader.py / data_augmentor.py
(資料載入、資料增強)
    │   ├── cnn_autoencoder.py / trainer.py
(CNN 模型定義、基礎訓練器)
    │   ├── semi_supervised_trainer.py / run_semi_supervise
d.py (半監督式訓練與執行腳本)
    │   ├── anomaly_scorer.py / threshold_tuner.py
(異常評分、閥值自動調校)
    │   ├── threshold_tuner_semi.py
```

```
(半監督式閥值調校)
    │   ├── run_training.py / run_full_pipeline.py
(單模訓練、全流程自動化腳本)
    │   ├── storage.py / session_manager.py / cleaner.py
(資料儲存、Session 管理、清理工具)
    │   ├── generate_attack_pcap.py
(測試用攻擊封包生成器)
    │   └── tests/ (test_parser.py / test_anomaly.py / test
_semi_supervised.py 等)
    │
    ├── analyzer/                    前端網頁 App
    │   ├── models.py                AnalysisSession / Alert
/ CNNResult (資料庫模型)
    │   ├── views.py                 儀表板 / 上傳 / Session
列表 / 詳情 / 即時
    │   ├── urls.py                  網頁路徑導航
    │   └── admin.py                 Django 後台管理配置
    │
    ├── api/                         REST API App
    │   ├── views.py                 /api/sessions/ /api/cn
n/analyze/ /api/ai/chat/ (後端介面)
    │   └── urls.py                  API 路由定義
    │
    ├── network_platform/            Django 專案配置
    │   ├── settings.py              (CNN_MODEL_PATH, 門檻值
等全域配置)
    │   └── urls.py                  根路由配置
    │
    ├── templates/                   HTML 模板
    │   ├── base/base.html           共用版型 (Sidebar + Tail
wind + Chart.js)
    │   └── analyzer/
    │       ├── dashboard.html       儀表板 (統計卡片 + 攻擊分佈
圖)
    │       ├── upload.html          PCAP 檔案上傳頁面
    │       ├── sessions.html        歷史分析紀錄列表
    │       ├── session_detail.html 分析結果詳情 + CNN 動態執
行畫面
```

```
    │        ├── live.html            即時監控介面 (前端輪詢 API)
    │        └── ai_chat.html        AI 安全助理對話視窗 (串接 n8n/Gemini)
    │
    ├── output/                       模型輸出與處理後的資料
    │   ├── dataset_cicids2017/       (.npy 格式的訓練/測試資料)
    │   ├── model/                    (標準模型、訓練曲線、混淆矩陣)
    │   └── model_semi/               (半監督式模型與評估報告)
    │
    ├── data/                         原始資料集 (如 cicids2017_cleaned.csv)
    ├── media/                        用戶上傳的 PCAP 與系統生成的 PDF 報告
    ├── static/                       靜態資源 (CSS / JS / Images)
    ├── requirements.txt              專案依賴套件表
    └── manage.py                     Django 管理入口
```

模型訓練相關程式實作

CNN Autoencoder 架構（core/cnn_autoencoder.py）

模型採用「編碼器－解碼器」非監督架構，訓練目標僅使用正常流量，讓模型學習重建正常封包的典型特徵。核心方法為 reconstruction_error()：逐樣本計算像素級 MSE，分數越高代表越可能是異常封包。

- Encoder：Conv(1→16→32→64→64) + MaxPool×3 + FC(1024→128→latent_dim)，將 32×32 封包影像壓縮為低維潛在向量
- Decoder：FC(latent_dim→128→1024) + ConvTranspose 上採樣×3 + Sigmoid，重建影像至 [0,1] 值域
- 為避免對小型資料集過度泛化，通道數從原版 (32→64→128→256) 縮減至 (16→32→64→64)，參數量從8M 降至約 300K

非監督訓練流程（core/trainer.py）

Trainer 類別使用純非監督方式（只以正常流量訓練），損失函數採 MSE Loss、優化器為 Adam + ReduceLROnPlateau、Early Stopping（patience=15）自動儲存最佳權重、梯度裁剪（max_norm=1.0）防止梯度爆炸

梯度爆炸 : 是指在反向傳播 (Backpropagation) 演算法中，當梯度從輸出層向輸入層傳播時，可能會變得異常的大

梯度爆炸會帶來什麼問題？

- 訓練不穩定： 極大的梯度會導致權重在一次更新中發生劇烈的變化，使得訓練過程變得非常不穩定，模型可能無法收斂
- 權重溢出 (NaN)： 過大的梯度可能導致權重的值超出計算機所能表示的範圍，從而出現 NaN (Not a Number) 的情況，使得訓練完全失敗
- 模型性能下降： 即使沒有導致權重溢出，不穩定的訓練也可能導致模型無法學習到有效的表示，從而影響最終的性能

(以上引述自 : https://vocus.cc/article/6831935dfd897800010cf353)

端對端訓練腳本（core/run_training.py /core/run_full_pipeline.py）

提供統一命令列介面，支援四種資料集（simulate / NSL-KDD / CIC-IDS2017 / CIC-DDoS2019），並整合資料擴增（**DataAugmentor**）、多資料集合併、閾值自動回寫至 **settings.py** 與完整評估報告輸出

加入半監督式學習 （core/semi_supervised_trainer.py）

採用半監督式學習的原因 :

經過與組員討論和參考github、文獻之後，發現以下問題:

原本的模型採用非監督式學習，訓練時僅輸入正常流量，期望模型學會重建正常封包，使攻擊封包因重建誤差偏高而被偵測。然而，實際評估時發現嚴重問題：TP=0、Recall=0，所有攻擊封包皆未被偵測，原因是模型對攻擊封包的重建誤差與正常封包幾乎相同，閾值無法區分兩者。

還有使用非監督式學習，那就沒有使用資料集的意義，因為只需要正常流量即可，與使用資料集訓練的方法產生矛盾。

採用半監督式學習的優點 :

- 純非監督式模型無法保證攻擊封包的重建誤差一定高於正常封包，CNN Autoencoder 有很好的泛化能力，即使從未見過攻擊封包，也可能將其重建得很好，導致正常/攻擊誤差完全重疊，閾值失去判別效力。
- 半監督式學習引入少量已標記的攻擊樣本，透過邊界損失（Margin Loss）明確要求模型對攻擊封包的重建誤差必須超過設定的 margin 值，強制拉大正常/攻擊誤差的分離度（Separability）。

<u>模型訓練策略：</u>
本系統採用兩階段訓練策略：非監督式學習 + 半監督式學習

- 階段 1 (非監督預訓練)：仍以純正常流量訓練 CNN Autoencoder，建立正常封包的重建基礎，保留非監督學習的核心優勢（不需大量標記資料）
- 階段2 (半監督微調)：引入少量（10~20%）標記攻擊流量的樣本，使用邊界損失推高攻擊封包的重建誤差，改善重構能力與分類準確度，在保持對未知攻擊敏感度的同時，有效減少誤報並提升對特定已知攻擊的辨識精準度。

邊界損失：

$$L_margin = mean(max(0, margin - MSE(x_attack, x_hat_attack)))$$

若攻擊樣本的重建誤差低於 margin 目標值，損失為正值，梯度會推高模型對攻擊封包的重建誤差；若誤差已超過 margin，損失歸零，不再更新。

$$\text{整體組合損失為：} L_total = \alpha \times L_recon_normal + \beta \times L_margin_attack$$

其中 α（預設 1.0）維護正常流量重建能力，β（預設 0.5）控制推高攻擊誤差的力道，margin（預設 0.05）建議設為正常誤差均值的 2~5 倍

重建誤差閾值調校

基礎閾值調校模組（core/threshold_tuner.py）

原本的 Trainer 只支援單一固定百分位數，ThresholdTuner 新增系統性掃描：
- scan_percentiles()：掃描 [80, 85, 90, 92, 95, 97, 99] 百分位，計算每個閾值的 TP/TN/FP/FN/Precision/Recall/F1/FPR
- find_best_threshold()：根據 metric 參數（f1/recall/precision）自動選最佳閾值

- plot_threshold_curve()：繪製「百分位數 vs P/R/F1」曲線，視覺化 trade-off 關係

閾值原理：以正常流量誤差的第 p 百分位數為閾值，意義為「(100-p)% 的正常封包被誤判為異常（誤報率）」，百分位越高閾值越嚴格。

半監督增強版閾值調校（core/threshold_tuner_semi.py）

SemiSupervisedThresholdTuner 繼承原版並新增：
- calibrate_with_labels()：在正常與攻擊誤差的全值域均勻取 200 個候選閾值（不受百分位粒度限制），額外計算 MCC（Matthews Correlation Coefficient）
- multi_strategy_report()：同時輸出三種場景的推薦閾值—策略 A（高偵測率）、策略 B（平衡 F1）、策略 C（低誤報 FPR ≤ 5%）
- sensitivity_analysis()：分析最佳閾值的「穩定區間」，識別微調對效能的靈敏度

模型訓練測試結果

以下是使用 cic - ddos2019 資料集訓練結果為例

(備註 :原本的 cic - ddos2019容量為30GB，因筆電效能可能無法負荷，因此改用較小型的 cic - ddos2019資料集(91.6MB) 做測試 。之後會做更進一步訓練)

指令 : python core/run_training.py --dataset cicddos2019 --data-dir data/cic-ddos2019 --epochs 10 --latent 32
輸出結果:

```
============================================================
  CNN Autoencoder 訓練流程（資料集：cicddos2019）
============================================================
  PyTorch 2.5.1+cu121
  CUDA 可用: True

[Step 1] 載入資料集：cicddos2019
  [CIC-DDoS2019] 掃描到 1 個 CSV 檔案
  [CIC-IDS2017] 載入 1 個 CSV 檔案
    讀取: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
      正常: 97,718  攻擊: 128,027
```

```
  [CIC-IDS2017] 使用: 正常 50,000  攻擊 30,000
  [CIC-IDS2017] 影像尺寸: (32, 32)
  正常流量: 50,000 筆  攻擊流量: 30,000 筆

[Step 2] 儲存影像矩陣
  [DatasetFactory] normal: output/dataset_cicddos2019\X_nor
mal.npy  shape=(50000, 32, 32)
  [DatasetFactory] attack: output/dataset_cicddos2019\X_att
ack.npy  shape=(30000, 32, 32)
  [DatasetFactory] X_all: output/dataset_cicddos2019\X_all.
npy  shape=(80000, 32, 32)
  [DatasetFactory] y_all: output/dataset_cicddos2019\y_all.
npy  shape=(80000,)

[Step 3] 訓練 CNN Autoencoder
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 359,329 (1.37 MB)

  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 10 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────
──
  1        0.003424       0.000165       1.00e-03     8.5
  2        0.000172       0.000094       1.00e-03     7.1
  3        0.000142       0.000085       1.00e-03     7.0
  4        0.000128       0.000084       1.00e-03     7.1
  5        0.000116       0.000089       1.00e-03     7.1
  6        0.000112       0.000087       1.00e-03     7.0
  7        0.000101       0.000085       1.00e-03     6.9
  8        0.000099       0.000099       1.00e-03     6.9
  9        0.000093       0.000070       1.00e-03     6.6
  10       0.000090       0.000070       1.00e-03     6.5
```

```
  訓練完成！總耗時: 70.9s，最佳驗證損失: 0.000070
C:\Users\user\Desktop\final_project\core\trainer.py:294: Fu
tureWarning: You are using `torch.load` with `weights_only=
False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malic
ious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/mai
n/SECURITY.md#untrusted-models for more details). In a futu
re release, the default value for `weights_only` will be fl
ipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are expli
citly allowlisted by the user via `torch.serialization.add_
safe_globals`. We recommend you start setting `weights_only
=True` for any use case where you don't have full control o
f the loaded file. Please open an issue on GitHub for any i
ssues related to this experimental feature.
  self.model.load_state_dict(torch.load(model_path, map_loc
ation=self.device))
  [Trainer] 訓練結果已儲存: output/model\training_result.json
  [圖表] 訓練曲線已儲存: output/model\training_curve.png

[Step 4] 計算異常偵測閾值

  [Threshold] 重建誤差統計:
    Min   : 0.000004
    Mean  : 0.000070
    Max   : 0.010327
    閾值 (95th percentile): 0.000206
  [圖表] 誤差分布圖已儲存: output/model\error_distribution.png

[Step 5] 繪製重建對比圖
  [圖表] 重建對比圖已儲存: output/model\reconstruction_sample
s.png

[Step 6] 模型效能評估
```

```
  ==================================================
  模型評估結果（閾值: 0.000206）
  ==================================================
  指標                                 值
  ──────────────────────────────────────
  Precision                    0.8839
  Recall（偵測率）                 0.6344
  F1 Score                     0.7386
  Accuracy                     0.8317
  FPR（誤報率）                    0.0500
  AUC-ROC                      0.8470
  ──────────────────────────────────────
  TP=19032  TN=47500  FP=2500  FN=10968
  ==================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png

  正常流量平均誤差: 0.000070
  攻擊流量平均誤差: 0.001752
  兩者誤差比值    : 24.86x
  [圖表] ROC/PR 曲線已儲存: output/model\roc_pr_curve.png
  [圖表] 分數分布圖已儲存: output/model\score_distribution.png

============================================================
=
  完成！輸出目錄: C:\Users\user\Desktop\final_project\output
\model
    best_model.pt                              1426.2 KB
    confusion_matrix.png                       33.3 KB
    error_distribution.png                     31.2 KB
    pipeline_report.json                       2.7 KB
    reconstruction_samples.png                 31.5 KB
    roc_pr_curve.png                           95.0 KB
    score_distribution.png                     44.1 KB
    threshold_curve.png                        80.3 KB
    training_curve.png                         43.6 KB
    training_result.json                       0.9 KB
============================================================
=
```

訓練 + 閾值微調 ：在訓練後自動尋找最佳的異常偵測閾值

(以cic - ddos2019 為例)

指令 : python core/run_threshold_tuning.py --dataset cicddos2019 --data-dir data/cic-ddos2019 --ablation --epochs 10

輸出結果:

```
============================================================
  模型訓練與閾值調校（資料集：cicddos2019）
============================================================
  PyTorch 2.5.1+cu121
  CUDA 可用: True

[Step 1] 載入資料集：cicddos2019

  [CIC-DDoS2019] 掃描到 1 個 CSV 檔案
  [CIC-IDS2017] 載入 1 個 CSV 檔案
    讀取: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
      正常: 97,718  攻擊: 128,027
  [CIC-IDS2017] 使用: 正常 50,000  攻擊 30,000
  [CIC-IDS2017] 影像尺寸: (32, 32)
  正常流量: 50,000 筆  攻擊流量: 30,000 筆
  [DatasetFactory] normal: output/dataset_cicddos2019\X_normal.npy  shape=(50000, 32, 32)
  [DatasetFactory] attack: output/dataset_cicddos2019\X_attack.npy  shape=(30000, 32, 32)
  [DatasetFactory] X_all: output/dataset_cicddos2019\X_all.npy  shape=(80000, 32, 32)
  [DatasetFactory] y_all: output/dataset_cicddos2019\y_all.npy  shape=(80000,)

[Step 2] 訓練 CNN Autoencoder
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 359,329 (1.37 MB)
```

```
  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 1 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────
───
  1        0.001979       0.000157       1.00e-03     7.3

  訓練完成！總耗時: 7.3s，最佳驗證損失: 0.000157
C:\Users\user\Desktop\final_project\core\trainer.py:294: Fu
tureWarning: You are using `torch.load` with `weights_only=
False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malic
ious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/mai
n/SECURITY.md#untrusted-models for more details). In a futu
re release, the default value for `weights_only` will be fl
ipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are expli
citly allowlisted by the user via `torch.serialization.add_
safe_globals`. We recommend you start setting `weights_only
=True` for any use case where you don't have full control o
f the loaded file. Please open an issue on GitHub for any i
ssues related to this experimental feature.
  self.model.load_state_dict(torch.load(model_path, map_loc
ation=self.device))
  [Trainer] 訓練結果已儲存: output/model\training_result.json
  [圖表] 訓練曲線已儲存: output/model\training_curve.png

[Step 3] 執行閾值調校分析 (ThresholdTuner)

  [ThresholdTuner] 計算重建誤差...
  正常流量誤差範圍: [0.000033, 0.010413]
```

```
  攻擊流量誤差範圍: [0.000039, 0.015826]
  攻擊 / 正常誤差比值: 15.48x

     百分位          閾值   Precision    Recall        F1
FPR
  ---------------------------------------------------------
-
      80th    0.000176      0.6565    0.6372    0.6467
0.2000
      85th    0.000229      0.7182    0.6372    0.6753
0.1500
      90th    0.000319      0.7925    0.6364    0.7059
0.1000
      92th    0.000381      0.8263    0.6341    0.7175
0.0800
      95th    0.000523      0.8831    0.6297    0.7352
0.0500
      97th    0.000849      0.9224    0.5946    0.7231
0.0300
      99th    0.001273      0.9677    0.5000    0.6594
0.0100

  [圖表] 閾值調校曲線已儲存: output/model\threshold_curve.png

[Step 4] 執行消融實驗 (Ablation Study)
  測試 latent_dim = 8...
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 353,161 (1.35 MB)

  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 20 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────
```

```
──
  1        0.003290       0.000238       1.00e-03     6.7
  2        0.000198       0.000124       1.00e-03     7.0
  3        0.000153       0.000102       1.00e-03     7.0
  4        0.000134       0.000093       1.00e-03     6.9
  5        0.000124       0.000082       1.00e-03     7.0
  6        0.000114       0.000084       1.00e-03     7.0
  7        0.000107       0.000070       1.00e-03     6.9
  8        0.000103       0.000079       1.00e-03     6.9
  9        0.000096       0.000065       1.00e-03     6.9
  10       0.000091       0.000070       1.00e-03     5.8
  11       0.000091       0.000085       1.00e-03     5.8
  12       0.000088       0.000063       1.00e-03     5.8
  13       0.000087       0.000055       1.00e-03     6.6
  14       0.000086       0.000064       1.00e-03     6.8
  15       0.000086       0.000057       1.00e-03     7.1
  16       0.000082       0.000065       1.00e-03     6.8
  17       0.000083       0.000065       1.00e-03     6.6
  18       0.000083       0.000063       1.00e-03     6.1
  19       0.000081       0.000073       1.00e-03     6.0
  20       0.000080       0.000055       1.00e-03     5.9

  訓練完成！總耗時: 131.7s，最佳驗證損失: 0.000055
  [Trainer] 訓練結果已儲存: output/ablation_ld_8\training_res
ult.json

  [Threshold] 重建誤差統計:
    Min   : 0.000003
    Mean  : 0.000055
    Max   : 0.010578
    閾值 (95th percentile): 0.000167
  [圖表] 誤差分布圖已儲存: output/ablation_ld_8\error_distribu
tion.png

  ====================================================
  模型評估結果（閾值: 0.000167）
  ====================================================
  指標                                    值
```

```
  ────────────────────────────────────────
  Precision                  0.8838
  Recall（偵測率）                 0.6338
  F1 Score                   0.7382
  Accuracy                   0.8314
  FPR（誤報率）                    0.0500
  AUC-ROC                    0.8784
  ────────────────────────────────────────
  TP=19013  TN=47500  FP=2500  FN=10987
  ==================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png
  測試 latent_dim = 16...
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 355,217 (1.36 MB)

  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 20 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────
──
  1        0.002507       0.000174       1.00e-03     6.7
  2        0.000175       0.000109       1.00e-03     6.1
  3        0.000142       0.000099       1.00e-03     6.6
  4        0.000128       0.000082       1.00e-03     6.4
  5        0.000117       0.000075       1.00e-03     6.1
  6        0.000110       0.000072       1.00e-03     6.0
  7        0.000102       0.000068       1.00e-03     6.0
  8        0.000100       0.000081       1.00e-03     5.8
  9        0.000092       0.000074       1.00e-03     6.1
  10       0.000095       0.000068       1.00e-03     6.0
  11       0.000091       0.000087       1.00e-03     6.0
  12       0.000089       0.000068       1.00e-03     5.8
  13       0.000089       0.000072       1.00e-03     5.7
```

```
  14       0.000087       0.000067       1.00e-03     5.6
  15       0.000086       0.000076       1.00e-03     6.2
  16       0.000087       0.000057       1.00e-03     6.5
  17       0.000084       0.000055       1.00e-03     7.0
  18       0.000081       0.000066       1.00e-03     7.0
  19       0.000082       0.000055       1.00e-03     6.8
  20       0.000080       0.000053       1.00e-03     6.7

  訓練完成！總耗時: 125.1s，最佳驗證損失: 0.000053
C:\Users\user\Desktop\final_project\core\trainer.py:294: Fu
tureWarning: You are using `torch.load` with `weights_only=
False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malic
ious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/mai
n/SECURITY.md#untrusted-models for more details). In a futu
re release, the default value for `weights_only` will be fl
ipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are expli
citly allowlisted by the user via `torch.serialization.add_
safe_globals`. We recommend you start setting `weights_only
=True` for any use case where you don't have full control o
f the loaded file. Please open an issue on GitHub for any i
ssues related to this experimental feature.
  self.model.load_state_dict(torch.load(model_path, map_loc
ation=self.device))
  [Trainer] 訓練結果已儲存: output/ablation_ld_16\training_re
sult.json

  [Threshold] 重建誤差統計:
    Min   : 0.000003
    Mean  : 0.000053
    Max   : 0.010176
    閾值 (95th percentile): 0.000156
  [圖表] 誤差分布圖已儲存: output/ablation_ld_16\error_distrib
ution.png
```

```
  ==================================================
  模型評估結果（閾值: 0.000156）
  ==================================================
  指標                                 值
  ────────────────────────────────────
  Precision                    0.8829
  Recall（偵測率）                 0.6280
  F1 Score                     0.7340
  Accuracy                     0.8293
  FPR（誤報率）                    0.0500
  AUC-ROC                      0.8437
  ────────────────────────────────────
  TP=18841  TN=47500  FP=2500  FN=11159
  ==================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png
  測試 latent_dim = 32...
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 359,329 (1.37 MB)

  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 20 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────
──
  1        0.002997       0.000136       1.00e-03     7.1
  2        0.000162       0.000116       1.00e-03     7.0
  3        0.000132       0.000099       1.00e-03     7.0
  4        0.000124       0.000117       1.00e-03     7.1
  5        0.000116       0.000197       1.00e-03     6.8
  6        0.000113       0.000082       1.00e-03     6.8
  7        0.000106       0.000071       1.00e-03     7.1
  8        0.000098       0.000074       1.00e-03     7.0
  9        0.000096       0.000068       1.00e-03     6.9
```

```
  10       0.000094       0.000070       1.00e-03     7.0
  11       0.000095       0.000072       1.00e-03     6.9
  12       0.000088       0.000090       1.00e-03     6.9
  13       0.000088       0.000058       1.00e-03     6.4
  14       0.000085       0.000055       1.00e-03     7.5
  15       0.000085       0.000073       1.00e-03     6.9
  16       0.000084       0.000069       1.00e-03     6.5
  17       0.000085       0.000074       1.00e-03     6.4
  18       0.000081       0.000071       1.00e-03     6.8
  19       0.000081       0.000062       1.00e-03     7.3
  20       0.000080       0.000064       1.00e-03     6.5

  訓練完成！總耗時: 137.9s，最佳驗證損失: 0.000055
C:\Users\user\Desktop\final_project\core\trainer.py:294: Fu
tureWarning: You are using `torch.load` with `weights_only=
False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malic
ious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/mai
n/SECURITY.md#untrusted-models for more details). In a futu
re release, the default value for `weights_only` will be fl
ipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are expli
citly allowlisted by the user via `torch.serialization.add_
safe_globals`. We recommend you start setting `weights_only
=True` for any use case where you don't have full control o
f the loaded file. Please open an issue on GitHub for any i
ssues related to this experimental feature.
  self.model.load_state_dict(torch.load(model_path, map_loc
ation=self.device))
  [Trainer] 訓練結果已儲存: output/ablation_ld_32\training_re
sult.json

  [Threshold] 重建誤差統計:
    Min   : 0.000003
    Mean  : 0.000055
    Max   : 0.009538
```

```
    閾值 (95th percentile): 0.000172
  [圖表] 誤差分布圖已儲存: output/ablation_ld_32\error_distrib
ution.png

  ==================================================
  模型評估結果（閾值: 0.000172）
  ==================================================
  指標                                 值
  ————————————————————————————————————
  Precision                    0.8832
  Recall（偵測率）                 0.6301
  F1 Score                     0.7355
  Accuracy                     0.8300
  FPR（誤報率）                    0.0500
  AUC-ROC                      0.8686
  ————————————————————————————————————
  TP=18903  TN=47500  FP=2500  FN=11097
  ==================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png
  測試 latent_dim = 64...
  [Trainer] 使用裝置: cuda
  [Trainer] 模型參數: 367,553 (1.40 MB)

  [Trainer] 載入資料: output/dataset_cicddos2019\X_normal.np
y
  [Trainer] 訓練集: 40000 個，驗證集: 10000 個
  [Trainer] 影像尺寸: torch.Size([1, 32, 32])

  [Trainer] 開始訓練（最多 20 epochs）
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ————————————————————————————————————————————————————————
___
  1        0.003745       0.001153       1.00e-03     6.2
  2        0.001138       0.001100       1.00e-03     6.1
  3        0.001118       0.001080       1.00e-03     6.6
  4        0.001103       0.001065       1.00e-03     6.5
  5        0.001092       0.001071       1.00e-03     6.6
```

```
  6         0.000577       0.000066       1.00e-03     7.4
  7         0.000104       0.000084       1.00e-03     7.7
  8         0.000098       0.000065       1.00e-03     6.7
  9         0.000093       0.000070       1.00e-03     6.4
  10        0.000089       0.000092       1.00e-03     7.1
  11        0.000087       0.000072       1.00e-03     7.3
  12        0.000086       0.000063       1.00e-03     7.1
  13        0.000086       0.000060       1.00e-03     7.2
  14        0.000088       0.000065       1.00e-03     7.2
  15        0.000082       0.000065       1.00e-03     7.2
  16        0.000085       0.000088       1.00e-03     7.5
  17        0.000082       0.000061       1.00e-03     7.4
  18        0.000078       0.000061       1.00e-03     6.9
  19        0.000082       0.000067       1.00e-03     9.7
  20        0.000080       0.000068       1.00e-03     7.0

  訓練完成！總耗時: 141.8s，最佳驗證損失: 0.000060
C:\Users\user\Desktop\final_project\core\trainer.py:294: Fu
tureWarning: You are using `torch.load` with `weights_only=
False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malic
ious pickle data which will execute arbitrary code during u
npickling (See https://github.com/pytorch/pytorch/blob/mai
n/SECURITY.md#untrusted-models for more details). In a futu
re release, the default value for `weights_only` will be fl
ipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are expli
citly allowlisted by the user via `torch.serialization.add_
safe_globals`. We recommend you start setting `weights_only
=True` for any use case where you don't have full control o
f the loaded file. Please open an issue on GitHub for any i
ssues related to this experimental feature.
  self.model.load_state_dict(torch.load(model_path, map_loc
ation=self.device))
  [Trainer] 訓練結果已儲存: output/ablation_ld_64\training_re
sult.json
```

```
  [Threshold] 重建誤差統計:
    Min   : 0.000003
    Mean  : 0.000060
    Max   : 0.010055
    閾值 (95th percentile): 0.000179
  [圖表] 誤差分布圖已儲存: output/ablation_ld_64\error_distrib
ution.png

  ====================================================
  模型評估結果（閾值: 0.000179）
  ====================================================
  指標                                 值
  ────────────────────────────────────
  Precision                    0.8800
  Recall（偵測率）                 0.6108
  F1 Score                     0.7211
  Accuracy                     0.8228
  FPR（誤報率）                    0.0500
  AUC-ROC                      0.8394
  ────────────────────────────────────
  TP=18325  TN=47500  FP=2500  FN=11675
  ====================================================
  [圖表] 混淆矩陣已儲存: output/model\confusion_matrix.png
  消融實驗完成，結果儲存至 output/model/ablation_results.png

============================================================
=
  閾值調校流程完成！
  報告輸出：C:\Users\user\Desktop\final_project\output\model
============================================================
=
```

半監督式學習訓練 : 強化已知攻擊辨識

(以cic - ddos2019 為例)

指令 : python core/run_semi_supervised.py --dataset cicddos2019 --data-dir data/cic-ddos2019 --pretrain-epochs 5 --finetune-epochs 5

輸出結果:

```
============================================================
=
  半監督式訓練流程（資料集：cicddos2019）
============================================================
=
  PyTorch 2.5.1+cu121
  CUDA 可用: True

[Step 1] 載入資料集：cicddos2019
  [CIC-DDoS2019] 掃描到 1 個 CSV 檔案
  [CIC-IDS2017] 載入 1 個 CSV 檔案
    讀取: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
      正常: 97,718  攻擊: 128,027
  [CIC-IDS2017] 使用: 正常 30,000  攻擊 20,000
  [CIC-IDS2017] 影像尺寸: (32, 32)
  訓練集: 24,000 正常 + 2,400 攻擊
  測試集: 6,000 正常 + 10,000 攻擊

[Step 3/4] 執行半監督訓練
  [SemiSupervisedTrainer] 使用裝置: cuda
  [SemiSupervisedTrainer] 訓練策略: Phase1(5 epochs) + Phase
2(5 epochs)
  [SemiSupervisedTrainer] 模型參數: 359,329 (1.37 MB)

  [SemiSupervisedTrainer] 載入資料
    正常流量: 24,000 筆
    攻擊流量: 2,400 筆（只取 20% 用於微調）
  [SemiSupervisedDataLoader] 資料分配:
    正常流量 - 訓練: 19,200，驗證: 4,800
    攻擊流量（標記）: 480 / 2,400 （20%）

============================================================
=
  Phase 1：無監督預訓練（共 5 epochs）
  損失函數：L = MSE(x, x_hat)  [只有正常流量]
============================================================
```

```
=
  Epoch    Train Loss     Val Loss       LR           時間
(s)
  ──────────────────────────────────────────────────────────────
─
  1        0.001985       0.000213       1.00e-03     21.3
  2        0.000228       0.000155       1.00e-03     3.4
  3        0.000167       0.000123       1.00e-03     3.7
  4        0.000153       0.000102       1.00e-03     3.4
  5        0.000144       0.000132       1.00e-03     3.6

  Phase 1 完成！最佳驗證損失: 0.000102
  預訓練模型已儲存: output/model_semi\pretrained_model.pt

============================================================
=
  Phase 2：半監督微調（共 5 epochs）
  損失函數：L = 1.0×L_recon + 0.5×L_margin
  邊界值（margin）: 0.1
============================================================
=
  Epoch    Normal Loss    Attack Loss    Total Loss     Val
Loss
  ──────────────────────────────────────────────────────────────
──────────────
  1        0.000393       0.018974       0.009880       0.0
01863     (6.7s)
  2        0.000136       0.000001       0.000136       0.0
00898     (6.4s)
  3        0.000109       0.000002       0.000110       0.0
00955     (5.8s)
  4        0.000113       0.000032       0.000129       0.0
01763     (5.8s)
  5        0.000096       0.000000       0.000096       0.0
01760     (5.9s)

  Phase 2 完成！最佳驗證損失: 0.000898
  最佳模型已儲存: output/model_semi\best_model.pt
```

```
  [圖表] 半監督訓練曲線已儲存: output/model_semi\semi_training_
curve.png

  [閾值計算] 方法: percentile
  正常流量誤差: mean=0.000922, std=0.001125
  攻擊流量誤差: mean=0.010799, std=0.004520
  分離比（攻擊/正常）: 11.71x
  閾值（95th 百分位）: 0.002955
  [SemiSupervisedTrainer] 訓練結果已儲存: output/model_semi\s
emi_training_result.json

[Step 5] 計算最佳閾值並評估

  [半監督結果]
    Precision: 0.9736
    Recall:    0.9998
    F1 Score:  0.9865
    AUC-ROC:   0.9978

============================================================
=
  半監督式訓練流程完成！
  輸出目錄：C:\Users\user\Desktop\final_project\output\model
_semi
============================================================
=
```

二、目前約完成 45 %之進度

比預計的進度 （☑️稍微 □嚴重）（□落後 ☑️超前）。

主要原因：考量到接近期中考試，因此前幾週有先做到本週一部分的實作

三、下次主要工作

(進度報告3) 消融實驗與模型壓縮評估，預計完成以下項目：

- 消融實驗：比較「有無欄位遮罩」、「不同影像尺寸（28×28/32×32/40×40）」、「不同潛在維度（8/16/32/64）」、「有無半監督微調」對 F1/AUC 的影響，並產出完整對比報告
- 模型壓縮：評估知識蒸餾（Knowledge Distillation）或剪枝（Pruning）對模型推論速度與準確率的影響。
- 基準比較：目前使用的半監督式與原本的非監督式 CNN Autoencoder 進行 Precision/Recall/F1/AUC 的比較。

四、遭遇問題(實作、技術或溝通問題等)

請描述遭遇問題/目前解決進度/解決方法/未解決原因：

1. 問題 : 小型資料集（< 500 筆）時，攻擊封包重建誤差與正常封包相近，閾值難以設定
   - 解決方法:
     - 模型參數縮減（4.8M→300K）強制學習精簡表示
     - 引入半監督損失函數(MarginLoss) 推高攻擊誤差
2. 問題 : CIC-IDS2017 CSV 標籤欄名含前置空格（" Label"）造成載入失敗
   - 解決方法: 在 find_label_column() 中以 strip().lower() 正規化比對，並擴充 possible_names 涵蓋常見命名
3. 半監督的階段2， 若 β 過大，會破壞階段1建立的正常流量重建能力
   - 階段2學習率降至 階段1 的 1/10；改用 CosineAnnealingLR；β 限制在 0.3~0.8；監控正常流量驗證損失，上升超 20% 即提前停止

     (CosineAnnealingLR : 餘弦退火學習率排程，用來動態調整學習率的策略)

五、老師建議

(包含學長姊的建議)

1. 聲音略小，可以更有自信些
2. 解釋的案例要符合專題的情境
3. 改善 reference 的格式撰寫
4. 頁面規劃內容跟圖表很小，需要做調整

5. Figma有學生版可以註冊看看
6. 為甚麼選擇max pooling

六、回覆建議

關於為何選擇Max Pooling (最大池化)，原因 :

1. 保留異常特徵而非平均特徵
   封包影像的異常通常體現在特定位元組位置出現不尋常的高數值（例如 SYN Flood 的 TCP flags 欄位、ARP Spoofing 的 MAC 區段）。Max Pooling 在每個區域只取最大值，能保留這些局部突出的異常訊號；若改用 Average Pooling，這些突出值會被周圍的零值稀釋，讓 Encoder 的潛在向量失去辨別力。

2. 配合欄位遮罩後大量補零的影像特性
   packet_visualizer.py 中，IP/Port 等欄位被遮罩為 0x00，短封包也以 0x00 補齊至 1024 bytes，使影像大部分區域是零值。Average Pooling 在零值密集區域會產生大量接近零的特徵圖，壓縮後的潛在向量會丟失稀疏訊號；Max Pooling 則能有效從稀疏的非零區域提取特徵。

3. 梯度稀疏性有助於後續 Grad-CAM
   anomaly_scorer.py 的 gradcam_packet() 方法透過對重建誤差反向傳播梯度來定位異常位元組。Max Pooling 的梯度只流向當初「贏得」最大值的位置（其餘梯度為零），使梯度圖更集中、更易解讀哪個區段造成了高重建誤差。Average Pooling 的梯度會均勻分散，產生模糊的熱力圖。

4. 計算效率
   Max Pooling 不含可學習參數，運算量比 Strided Convolution 少，對這個約 300K 參數的輕量模型來說能節省訓練時間，且在小型資料集（< 500 筆）上較不容易因參數過多而過擬合。